Shared-Memory SPSC IPC and io_uring Wakeup Modes: A Measurement Study of Wakeup Mechanisms and Measurement Controls

Rahul Raman Yuvraj Deshmukh B. Thangaraju
Department of Computer Science and Engineering
International Institute of Information Technology Bangalore (IIITB), Bangalore, India
rahul.raman@iiitb.ac.in yuvraj.deshmukh@iiitb.ac.in b.thangaraju@iiitb.ac.in

***Abstract*—Inter-Process Communication (IPC) performance is a foundational bottleneck in concurrent, multi-process software systems. Traditional kernel-mediated mechanisms—POSIX Pipes, UNIX Domain Sockets, POSIX Message Queues—impose mandatory system calls, context switches, and user-to-kernel memory copying on every message, creating an irreducible per-message overhead of several microseconds. Modern high-performance systems reduce this cost by employing lock-free Single-Producer Single-Consumer (SPSC) shared-memory ring buffers that transfer payload data entirely in userspace via memcpy into POSIX shared memory regions. A critical and largely unanswered question, however, is: when the shared ring empties and the consumer must sleep, which wakeup primitive should be used—and what does Linux's io_uring framework specifically contribute over simpler alternatives such as futex or eventfd?**

This paper presents a rigorous empirical measurement study that decouples payload transport from wakeup-mechanism signaling and applies a two-suite evaluation framework: (1) a wakeup-mechanism ablation comparing six coordination strategies (busy_poll, spin_backoff, adaptive, futex, eventfd, io_uring) on an identical cache-aligned SPSC ring under saturated and bursty traffic conditions across a 64 B–1 MiB message-size sweep; and (2) a corrected depth-1 ping-pong latency suite in which both the send and receive timestamps are recorded on the single initiator core using CLOCK_MONOTONIC_RAW, eliminating cross-core TSC drift and in-flight pipelining artefacts, reporting full percentile distributions (P50, P90, P99, P99.9) over 100 000 round-trips per configuration.

Our results show that the shared-memory ring delivers the primary performance gains. Spinning variants achieve median single-trip latencies of 345–381 ns at 64 B—over a 15× improvement over pipe or socket—and peak direct-ring streaming throughput of 28.17 GiB/s at 64 KiB. In depth-1 ping-pong tests (Suite 2), io_uring incurs a modest per-event ring submission overhead (+2.8 μs over futex at 64 B). However, under bursty wakeup conditions (Suite 1), where wakeups occur when the ring empties, io_uring performs on par with futex and eventfd (≈1.09–1.10 ms median wakeup latency with ≈0 % idle CPU consumption). The shared-memory ring, rather than the wakeup primitive, delivers the primary gain. A frequency-pinned replication (`performance` governor and `no_turbo=1`) reproduces the close ordering of the kernel-sleeping variants. We additionally evaluate SQPOLL as a separate configuration: its 64 B bursty wakeup median (1058.5 μs) is close to interrupt-mode io_uring (1095.0 μs), rather than a decisive improvement.

***Index Terms*—io_uring, IPC, shared memory, SPSC ring buffer, lock-free, wakeup mechanism, latency, throughput, Linux kernel, systems performance**

I. Introduction

Modern software infrastructure—spanning high-frequency trading (HFT) matching engines, real-time database kernels, media processing pipelines, and containerised microservice mesh proxies—relies heavily on efficient Inter-Process Communication. Even a modest per-message overhead of a few microseconds accumulates to tens of milliseconds when millions of messages flow per second. At 10 Mop/s, a 3 μs per-message syscall overhead alone consumes 30 % of the CPU budget.

Linux provides a rich set of mature IPC primitives. POSIX FIFOs and UNIX domain sockets route all data through the kernel, imposing context switches and double memory copies on every message. POSIX message queues add priority ordering but retain kernel-mediated delivery semantics. Shared memory avoids the data-copy penalty by mapping identical physical pages into both producer and consumer address spaces; the kernel is contacted only for initial setup.

The cost of these kernel interactions is non-trivial and grows with message rate. A `write`/`read` pair on a POSIX pipe costs approximately 1–3 μs in system-call overhead on modern Linux, independent of payload size. For a system exchanging 5 Mmsg/s, this overhead alone consumes a full CPU core. Beyond raw syscall cost, kernel buffers are allocated from a different NUMA domain relative to the application heap on multi-socket machines, adding DRAM access latency to every message. Real-time systems with sub-1 ms SLAs cannot tolerate this overhead when message rates exceed a few hundred thousand per second.

Lock-free SPSC ring buffers—popularised by the LMAX Disruptor [5] and Intel DPDK [6]—take shared memory a step further by coordinating access using only atomic load/store instructions on cache-line-aligned indices, avoiding any kernel involvement on the data path. Message delivery in the uncontended case reduces to a `memcpy` plus two atomic index updates, costing roughly 200 ns at 64 B—an order of magnitude below any kernel-mediated mechanism.

A remaining challenge is the *empty-ring wakeup problem*: when the ring is empty, busy-spinning wastes a full CPU core; sleeping the consumer requires a kernel-level wakeup mechanism.

Linux's `io_uring` interface (introduced in kernel 5.1 [1]) has attracted substantial interest for its promise of zero-syscall I/O. A growing body of practitioner literature presents "`io_uring`-based IPC rings" as offering superior latency and throughput over both traditional IPC and simpler shared-memory approaches. However, these claims conflate two independent design decisions: (1) the payload data path choice (shared memory vs. kernel-copy), and (2) the wakeup mechanism choice (`io_uring` vs. `futex` vs. spin). No prior measurement study isolates these dimensions on identical hardware with a controlled ablation.

Contributions

1) **Controlled wakeup ablation.** Six pluggable wakeup strategies are implemented on a bit-identical SPSC ring buffer; their impact on latency, throughput, CPU utilization, and syscall rate is measured across a 64 B–1 MiB payload sweep.
2) **Corrected depth-1 ping-pong protocol.** Queue depth is enforced at 1 and both round-trip endpoints are timestamped on the same core with `CLOCK_MONOTONIC_RAW`, eliminating TSC skew and pipelining artefacts.
3) **Tail-latency characterisation.** P50, P90, P99, P99.9, and 95 % confidence intervals are reported for every supported payload size, including 64 KiB (10 000 rounds) and 1 MiB (2 000 rounds). The 64 B and 4 KiB configurations use 100 000 and 50 000 rounds, respectively.
4) **Explicit measurement scope.** We report interrupt-mode and SQPOLL io_uring as distinct configurations, and record fixed-frequency settings for the low-load wakeup measurements.
5) **Reproducible artifact.** All source code, raw CSV data, and figure-generation scripts are published at https://github.com/Rahul09123/io_uring-IPC.

II. Background & Related Work

A. Traditional Kernel-Mediated IPC

POSIX Pipes (FIFOs) (Stevens and Rago [2]) provide uni-directional byte streams backed by a kernel circular buffer (default 64 KiB, tunable to 1 MiB via `fcntl(F_SETPIPE_SZ)`). Every `write` copies data from user to kernel space; every `read` copies it back. The producer and consumer alternate between running and sleeping states as the pipe buffer fills and drains, incurring scheduler-wakeup latency on each transition. At very small messages (64 B), the pipe's fixed per-syscall entry cost dominates; at large messages ($\geq$256 KiB), buffer-saturation and block-layer flushing limit throughput to $\approx$5–6 GiB/s regardless of CPU speed.

UNIX Domain Sockets bypass the network stack but retain the socket buffer abstraction (`sk_buff`). Each `sendmsg`/`recvmsg` pair requires a system call and a socket-layer serialisation lock (`mutex_lock` visible in flamegraph traces). Socket send/receive buffers are tuned to 2 MiB via `SO_SNDBUF`/`SO_RCVBUF` in our implementation. Despite the per-call overhead, UNIX sockets achieve competitive throughput at large payload sizes because the kernel socket buffer is not bounded by a fixed slot count and can leverage zero-copy page-flipping for contiguous large writes.

POSIX Message Queues (`mqueue`) deliver structured, priority-sorted messages via a virtual filesystem inode under `/dev/mqueue`. A key optimisation—`pipelined_send`—delivers directly into a blocked receiver's address space when one is already waiting, bypassing intermediate queue staging and making `mqueue` competitive with shared-memory rings at small message sizes. However, the default maximum message size is 8 KiB and the maximum queue depth is 10 messages, both controlled by kernel tunables (`fs.mqueue.msgsize_max`, `fs.mqueue.msg_max`), imposing hard architectural limits at scale.

TABLE I
Kernel-IPC System-Call and Copy Costs per Message

Mechanism	Syscalls /msg	Copies /msg	Kernel buffer
POSIX Pipe	2	2	Ring buf (64 KiB–1 MiB)
UNIX Socket	2	2	`sk_buff` (up to 2 MiB)
POSIX MQ	2	1–2	VFS inode (10 msgs)
SHM Ring	0	1	None (userspace)

B. Lock-Free SPSC Shared-Memory Rings

Shared-memory IPC maps identical physical pages into producer and consumer address spaces via `shm_open` and `mmap`. SPSC ring buffers (Lamport [4], Herlihy and Shavit [9], LMAX Disruptor [5], DPDK `rte_ring` [6]) coordinate using a pair of atomic integer indices. The critical correctness rule is: the producer advances `head` with `release` ordering after writing payload; the consumer reads `head` with `acquire` ordering before consuming.

The C++17 memory model formally defines these orderings. A *release store* ensures all prior writes are visible to any thread that subsequently performs an *acquire load* of the same variable. For an SPSC ring, this means the consumer observing `head > tail` is guaranteed to see the complete payload written by the producer before `head` was advanced. Using `memory_order_relaxed` on `head` (a common optimisation) works on x86 due to Total Store Order (TSO) but is technically undefined behaviour on weakly-ordered architectures (ARM, RISC-V, Power) that permit Store-Load reordering. We use `memory_order_seq_cst` on the publish store to maintain portability and prevent the lost-wakeup race.

False sharing—where producer and consumer cache lines collide—is prevented by placing `head`, `tail`, and control flags on separate 64-byte-aligned cache lines using `alignas(64)`. Without this separation, a producer store to `head` invalidates

the cache line containing `tail` on the consumer core, forcing a cache-coherency RFO (Request For Ownership) on every slot exchange and doubling effective memory latency.

C. The io_uring Interface

Introduced by Jens Axboe [1] and merged in Linux 5.1, `io_uring` exposes two ring buffers shared between userspace and the kernel: the Submission Queue (SQ) and the Completion Queue (CQ). In the default interrupt-driven mode (`flags=0`), a single `io_uring_enter` system call submits all pending SQEs and optionally waits for completions. In SQPOLL mode (`IORING_SETUP_SQPOLL`), a dedicated kernel poller thread eliminates the syscall entirely but requires elevated privileges.

In our implementation, `io_uring` is used *exclusively for the wakeup signaling path*. When the consumer sleeps, it submits an `IORING_OP_READ` on a named FIFO and blocks in `io_uring_submit_and_wait`. When the producer detects `consumer_sleeping = 1`, it submits an `IORING_OP_WRITE` to the same FIFO. The payload data path is pure `memcpy` into shared memory—`io_uring` never touches message bytes.

D. Prior Work on io_uring Performance

Didona et al. [7] compared `libaio`, SPDK, and `io_uring` for storage I/O, demonstrating that polling makes `io_uring` competitive with SPDK for sequential workloads. Ren and Trivedi [8] demonstrated that `io_uring` can match kernel-bypass performance for certain NVMe access patterns. Neither study addresses the question of wakeup-primitive selection for shared-memory IPC, leaving this dimension unexplored. Our work fills this gap with a controlled ablation study on a single fixed ring design.

III. System Design

Our architecture cleanly separates the data path from the control path.

A. Lock-Free SPSC Ring Buffer Layout

The shared ring buffer is a single POSIX shared memory object opened via `shm_open` and mapped with `mmap` into both the producer and consumer address spaces.

Listing 1. Cache-aligned SPSC RingBuffer (common.h)

```
struct alignas(64) RingBuffer {
  alignas(64) atomic<uint64_t> head;   // Producer write
     index
  alignas(64) atomic<uint64_t> tail;   // Consumer read
     index
  alignas(64) atomic<uint32_t> consumer_sleeping;

  struct alignas(64) Slot {
    uint64_t send_ns;           // Producer timestamp
     (CLOCK_MONOTONIC_RAW)
    uint64_t wakeup_publish_ns; // Wakeup-signal timestamp
    uint32_t size;              // Payload bytes written
    char     data[MAX_PAYLOAD]; // Up to 1 MiB of payload
  };
  Slot slots[NUM_SLOTS];        // Fixed 64-slot ring
     (~64 MiB total)
};
```

Three design properties are critical.

(1) Cache-line isolation. `head`, `tail`, and `consumer_sleeping` each occupy their own 64-byte cache line. Without this separation, a producer write to `head` invalidates the cache line containing `tail` on the consumer core, forcing an RFO (Request For Ownership) round-trip on every slot transition. Our hardware counter data (Table X) confirms this: the SHM Ring's 4.97 % LLC miss rate compares favourably to 30–34 % for mechanisms that traverse kernel buffers and inadvertently evict ring control variables.

(2) Sequential consistency at publish boundaries. `head` is stored with `memory_order_seq_cst` to prevent Store-Load reordering that could cause the producer to read stale `consumer_sleeping = 0` and skip sending a wakeup, silently deadlocking the consumer. On x86 (TSO), this is equivalent to a plain store followed by a `mfence`; on ARM64 it compiles to a `stlr` (store-release) instruction.

(3) Lost-wakeup prevention. A classic race exists in any producer-consumer pair: the consumer checks `head == tail` (empty), decides to sleep, but before setting `consumer_sleeping`, the producer writes a slot and checks `consumer_sleeping = 0` (decides not to signal), then the consumer sets `consumer_sleeping = 1` and waits—forever. We eliminate this race by having the consumer set `consumer_sleeping = 1` *first*, then re-check `head != tail` under a full memory fence. If a slot arrived in the window, the re-check catches it and the consumer proceeds without sleeping.

Data-flow walkthrough. On every message transfer: (a) the producer calls `memcpy` into `slots[head % NUM_SLOTS].data`, writes the timestamp and size, then executes a seq_cst store to `head`; (b) the consumer spin-reads `head` with acquire ordering; once it observes `head > tail`, it processes the slot and advances `tail` with a release store; (c) if the consumer finds the ring empty after its deadlock-prevention recheck, it invokes the configured wakeup primitive to sleep. Steps (a) and (b) involve zero system calls; step (c) invokes at most one system call per wakeup event.

B. Pluggable Wakeup Layer: Six Ablation Variants

To isolate the contribution of the wakeup mechanism, we implement six interchangeable wakeup modules on a single, bit-identical SPSC ring buffer data structure. Each primitive is invoked via its standard Linux kernel system interface (Linux man-pages [10]): Only the functions `consumer_wait()` and `producer_signal()` differ, guaranteeing that any observed latency difference is attributable solely to the wakeup mechanism.

a) 1. `busy_poll`.: The consumer spins in a tight `while(head.load(relaxed) == tail)` loop with no backoff. Zero system calls are issued. Latency is minimal (≈200 ns) but one CPU core is permanently pegged at 100 %.

b) 2. `spin_backoff`.: Same tight spin with an `_mm_pause()` intrinsic on every iteration. `_mm_pause` inserts a brief pipeline delay on x86, reducing memory-ordering

storms and lowering power by ≈20 % vs. busy_poll with negligible latency impact.

c) 3. adaptive.: The consumer spins for up to 500 iterations; if no data arrives, it falls back to a futex sleep. This hybrid targets low-latency workloads where the ring is rarely empty (spin wins) while gracefully handling burst-idle traffic (futex sleep saves power).

d) 4. futex.: The consumer stores 1 to consumer_sleeping and calls SYS_futex(FUTEX_WAIT). The kernel checks the address atomically; if it still reads 1, the thread is suspended. The producer stores 0 and calls FUTEX_WAKE. One system call per sleep and per wakeup event.

e) 5. eventfd.: The consumer polls an eventfd file descriptor and then reads to consume the wakeup token. The producer writes 1 to the eventfd. The eventfd is compatible with epoll/select, easing integration into existing event-loop frameworks.

f) 6. io_uring.: The consumer submits an IORING_OP_READ SQE targeting a named FIFO (/tmp/uring_sig_fifo) and blocks in io_uring_submit_and_wait(ring, 1). The producer submits an IORING_OP_WRITE SQE to the same FIFO and calls io_uring_submit. The io_uring context can simultaneously monitor additional FDs, making it advantageous for frameworks managing many concurrent channels. Our primary ablation evaluates standard interrupt mode (flags=0); additionally, the implementation provides runtime support for IORING_SETUP_SQPOLL (SQPOLL mode via USE_SQPOLL=1), which offloads ring polling to an asynchronous kernel thread under CAP_SYS_ADMIN.

TABLE II
Wakeup Mechanism Properties Summary

Variant	Wait Strategy	Idle CPU	Sys/wake
busy_poll	Tight spin	100 %	0
spin_back.	Spin+PAUSE	High	0
adaptive	Spin-then-futex	Low	0–1
futex	FUTEX_WAIT	≈0 %	2
eventfd	poll+read	≈0 %	2
io_uring	sub._and_wait	≈0 %	2

IV. Experimental Methodology

A. Two-Suite Benchmark Framework

Suite 1 — Wakeup-Mechanism Ablation. The producer streams messages continuously under two regimes: *saturated* (back-to-back sends; ring rarely empties) and *bursty* (64-message burst, then 1 ms sleep; ring empties every burst). The bursty regime directly exposes wakeup primitive latency. Payload sizes sweep across {64 B, 256 B, 1 KiB, 4 KiB, 64 KiB, 1 MiB}. $N = 15$ measured runs per configuration (one warmup discarded).

Suite 2 — Corrected Depth-1 Ping-Pong. Queue depth is strictly enforced to 1: the initiator sends one message and blocks waiting for the echo server to reflect it before sending the next. Both t_{start} and t_{end} are recorded on **Initiator Core 1** using clock_gettime(CLOCK_MONOTONIC_RAW). Single-trip latency:

$$L_{\text{trip}} = \frac{t_{\text{end}} - t_{\text{start}}}{2} \quad [\mu\text{s}]$$

100 000 round-trips per configuration; 1 000 warmup trips discarded. P50, P90, P99, and P99.9 percentiles reported.

B. Hardware and System Configuration

TABLE III
Reference Test Environment

Parameter	Value
System	ASUS Vivobook K3605ZF
OS	Ubuntu 24.04.4 LTS (kernel 6.x)
CPU	Intel Core i5-12500H (Alder Lake, 12th Gen)
Architecture	Hybrid P/E-core
Cores/Threads	12 Physical / 16 Logical
Frequency	400–4500 MHz (boost)
L1d Cache	448 KiB (12 instances)
L2 Cache	9 MiB (6 instances)
L3 Cache	18 MiB (shared)
RAM	16 GiB DDR4-3200
Storage	512 GB Samsung NVMe SSD
Compiler	g++ -O2, C++17
liburing	System package, Ubuntu 24.04
CPU Governor	performance; intel_pstate.no_turbo=1

Core affinity. Producer/Initiator pinned to Core 1; Consumer/Echo-Server pinned to Core 2 via sched_setaffinity. Both cores share the 18 MiB L3 cache, keeping the ring buffer warm across the inter-process boundary.

CPU tuning and frequency-pinned replication. The final ablation and ping-pong runs were collected on the native Linux host with Core 1 and Core 2 pinned, the performance governor on both cores, and intel_pstate.no_turbo=1. These settings are recorded in data/environment_ablation.txt and data/environment_pingpong.txt. Core isolation through isolcpus/nohz_full was not used; residual scheduler and idle-state effects therefore remain a limitation.

C. Statistical Methodology

For each (mechanism, payload size) pair we compute:

$$\bar{T} = \frac{1}{N}\sum_{k=1}^{N} T_k \ \ [\text{GiB/s}], \quad \bar{L} = \frac{1}{M}\sum_{i=1}^{M} L_i \ \ [\mu\text{s}]$$

$$\sigma_L = \sqrt{\frac{1}{M}\sum_{i=1}^{M}(L_i - \bar{L})^2}$$

where $N = 15$ is the run count and M is messages per run. 95 % confidence intervals use the Student-t approximation on run-level means.

D. Workload Volumes

TABLE IV
Workload Volumes per (Mechanism, Size) Configuration

Mechanism	≤1 KiB	4–64 KiB	≥256 KiB
POSIX Pipe	32 MiB	256 MiB	2 GiB
POSIX MQ	32 MiB	256 MiB	2 GiB
UNIX Socket	2 GiB	2 GiB	2 GiB
SHM Ring	2 GiB	2 GiB	2 GiB

V. Empirical Results

A. Unloaded Ping-Pong Latency (Suite 2)

Table V reports single-trip latency distributions (P50, P90, P99, P99.9, and mean ± 95% CI) at all four message sizes. Values are drawn consistently from the root `data/pingpong_*_summary.csv` files; legacy duplicates and the conflicting aggregate file are not used.

TABLE V
Single-Trip Ping-Pong Latency Distributions (μs). Each row reports P50, P90, P99, P99.9, and mean ± 95% CI from the canonical root summary CSVs.

Transport / Variant	P50	P90	P99	P99.9	Mean ± CI
Message Size: 64 B — lower is better					
`shm (adaptive)`	**0.345**	0.439	0.505	1.189	0.363 ± 0.002
`shm (busy_poll)`	0.381	0.500	0.851	1.584	0.414 ± 0.003
`shm (spin_back)`	0.727	1.284	1.317	1.823	0.846 ± 0.002
`shm (futex)`	5.001	5.097	6.395	8.872	5.033 ± 0.002
`shm (eventfd)`	5.039	5.169	6.444	8.832	5.081 ± 0.002
`pipe`	5.290	5.400	7.016	10.984	5.345 ± 0.005
`posix_mq`	5.567	5.685	7.361	9.824	5.629 ± 0.002
`unix_socket`	5.496	7.214	8.196	11.330	5.900 ± 0.017
`shm (io_uring)`	8.150	8.426	10.351	14.347	8.236 ± 0.003
Message Size: 4 KiB — lower is better					
`shm (adaptive)`	**2.263**	2.339	2.422	5.072	2.271 ± 0.001
`shm (busy_poll)`	2.306	2.413	3.576	8.527	2.353 ± 0.004
`shm (spin_back)`	2.663	2.724	3.120	5.494	2.656 ± 0.002
`shm (futex)`	6.735	6.899	8.575	10.677	6.808 ± 0.003
`shm (eventfd)`	6.809	7.137	8.403	10.874	6.898 ± 0.011
`pipe`	7.147	7.470	8.825	12.966	7.248 ± 0.004
`posix_mq`	8.746	9.028	10.493	13.275	8.813 ± 0.004
`shm (io_uring)`	9.855	10.119	12.056	15.679	9.923 ± 0.005
`unix_socket`	11.864	12.657	13.609	19.349	11.093 ± 0.016
Message Size: 64 KiB — lower is better					
`shm (busy_poll)`	**23.293**	23.845	26.982	48.473	23.280 ± 0.038
`shm (adaptive)`	23.888	24.006	25.829	27.300	23.858 ± 0.013
`shm (spin_back)`	24.204	24.298	25.881	27.637	24.187 ± 0.015
`shm (eventfd)`	27.244	27.478	29.342	31.348	27.228 ± 0.014
`shm (futex)`	27.227	27.464	29.242	31.836	27.216 ± 0.019
`unix_socket`	28.284	29.270	32.880	37.986	28.362 ± 0.022
`shm (io_uring)`	29.409	29.689	31.802	33.743	29.262 ± 0.019
`pipe`	35.861	36.380	40.106	41.696	35.935 ± 0.017
`posix_mq`	35.869	36.721	39.651	42.714	35.988 ± 0.038
Message Size: 1 MiB — lower is better					
`unix_socket`	**241.094**	248.331	261.743	365.375	242.922 ± 0.291
`shm (busy_poll)`	267.243	276.180	285.295	295.643	269.615 ± 0.205
`shm (spin_back)`	267.366	276.029	285.168	297.980	269.596 ± 0.203
`shm (eventfd)`	276.786	283.946	297.494	327.397	278.874 ± 0.316
`shm (adaptive)`	278.976	288.360	300.819	314.115	282.830 ± 2.696
`shm (futex)`	278.034	286.596	296.704	335.171	280.502 ± 0.236
`shm (io_uring)`	281.154	289.455	301.202	335.805	283.185 ± 0.256
`pipe`	473.293	482.965	542.826	1429.330	479.538 ± 3.410
`posix_mq`	N/A	N/A	N/A	N/A	N/A

Key observations:

(1) 13.6× spinning advantage at small messages. At 64 B, `adaptive` (0.345 μs) and `busy_poll` (0.381 μs) are over 13× lower than any kernel-sleeping variant. This gap represents scheduler wakeup latency: transitioning a sleeping thread to running requires dequeueing, CPU selection, and context switching—costing 2–4 μs on a loaded Linux system.

(2) Kernel-sleeping variants converge regardless of primitive. `futex` (5.001 μs), `eventfd` (5.039 μs), and `io_uring` (8.150 μs) all converge to 5.0–8.2 μs P50 at 64 B. The slight elevation of `io_uring` reflects SQ/CQ ring entry overhead. (These figures are recomputed from the full 100,000-trip raw distribution underlying Table V; the `io_uring` value differs marginally from earlier aggregate-based estimates reported in preliminary analysis.)

(3) POSIX MQ is valid through 64 KiB in the recorded run. The repository's 64 KiB summary records 10,000 round trips (P50 36.668 μs). The 256 KiB and 1 MiB rows are explicitly marked unsupported by the host POSIX-MQ message-size limit and are reported as N/A.

(4) UNIX Socket wins at 1 MiB. 242.90 μs vs. 270.9–281.3 μs for SHM variants. The kernel socket buffer handles large writes via page-flipping, bypassing the fixed-slot recycling stall that limits the SHM ring.

B. Ablation: Wakeup Variant Comparison (Suite 2)

At 1 MiB, all six SHM variants converge near 271–281 μs (Table V). Copying 1 MiB across the ring dominates all wakeup mechanism overhead, making wakeup selection irrelevant at that message size. Wakeup mechanism matters only when message service time is short enough to be latency-dominated.

C. Streaming Throughput: Saturated Regime (Suite 1)

Table VI presents mean throughput; Figure 2 visualises the full size sweep.

TABLE VI
Streaming Throughput (GiB/s) — Saturated Regime, $N = 15$ Runs

Wakeup Variant	64 B	4 KiB	64 KiB	1 MiB
`busy_poll`	0.316	10.17	**15.75**	**8.65**
`spin_backoff`	**0.344**	10.16	14.54	8.61
`adaptive`	0.338	10.43	15.53	8.36
`eventfd`	0.262	9.54	13.74	8.37
`futex`	0.300	**10.68**	15.60	8.18
`io_uring`	0.332	9.92	13.37	8.12

Key observations:

(1) Saturated throughput is primarily data-path bound. Under fully saturated load the ring rarely empties, so the wakeup primitive is rarely invoked. The six variants remain in the same order of magnitude, with the refreshed 64 KiB results spanning 13.37–15.75 GiB/s.

(2) Direct-ring baseline remains faster than the pipe baseline. The separately measured direct SHM+io_uring baseline reaches 28.17 GiB/s at 64 KiB, versus 5.07 GiB/s for POSIX pipe. This gain originates from eliminating kernel copies and context switches on the data path.

(3) Large messages converge. At 1 MiB the six refreshed variants span 8.12–8.65 GiB/s. Payload copying and the fixed 64-slot ring dominate the wakeup mechanism at this size.

(4) Small-message io_uring overhead at 64 B. At 64 B payload, the refreshed variants span 0.262–0.344 GiB/s. This

variation reflects fixed per-message coordination overhead. At 4 KiB and above, payload copying dominates the observed throughput range.

D. Bursty Regime: Pure Wakeup Latency Ablation (Suite 1)

In the bursty regime, the consumer empties the ring and enters a sleep state before each burst. Table VII isolates wakeup latency per variant.

TABLE VII
Bursty-Regime Wakeup and End-to-End Latencies at 64 B

Variant	E2E P50	Wakeup P50	Idle CPU	Syscalls
busy_poll	0.13 μs	**0.022** μs	100 %	0
spin_backoff	0.13 μs	0.094 μs	High	0
adaptive	135.8 μs	1028.9 μs	Low	0.01
io_uring	151.0 μs	1058.5 μs	≈0 %	0.02
futex	166.7 μs	1088.8 μs	≈0 %	0.02
eventfd	166.4 μs	1097.2 μs	≈0 %	0.02

TABLE VIII
CPU Resource Efficiency per 64 B Message (Bursty Regime)

Variant	Cycles / Msg	Ctx-Sw / Msg	Insn / Msg
futex	9,310	0.0263	8,512
eventfd	9,653	0.0280	8,711
io_uring	11,338	0.0287	10,928
adaptive	21,620	0.0243	12,323
busy_poll	114,789	0.0003	216,267
spin_backoff	202,011	0.0003	158,135

Table VIII quantifies the CPU cost side of the latency/CPU trade-off already visible qualitatively in Table VII. The kernel-sleeping variants (futex, eventfd, iouring) cost 9,300–11,300 cycles/message—roughly an order of magnitude less than the spinning variants (114,789–202,011 cycles/msg), because a sleeping thread consumes no cycles while blocked, whereas a spinning thread burns cycles continuously regardless of whether new data has arrived.

Note that spinbackoff's higher cycle count relative to busypoll (202,011 vs. 114,789) does not contradict its power-saving design intent from the spin-backoff design: the _mm_pause intrinsic inserted on every iteration deliberately reduces instructions retired per cycle (158,135 vs. 216,267 insn/msg, i.e. IPC $\approx$ 0.78 vs. $\approx$ 1.88), which is what actually lowers power draw at the microarchitectural level—cycle count and power are not the same axis. adaptive sits at an intermediate 21,620 cycles/msg, reflecting its spin-then-block hybrid: most cycles come from the initial spin window, with the fallback futex wait contributing comparatively little once the ring is confirmed empty.

Frequency-controlled interpretation. The final bursty and offered-load runs were collected with the performance governor and Turbo disabled. The approximately millisecond-scale low-load values therefore remain part of the measured sleep/wakeup path rather than an unrecorded DVFS artifact. The close values for futex, eventfd, and interrupt-mode io_uring support the limited conclusion that no configuration has a decisive advantage for a single SPSC sleeping wakeup on this platform.

SQPOLL check. SQPOLL was run as a separately labelled configuration. At 64 B its bursty wakeup P50 was 1058.5 μs, versus 1095.0 μs for interrupt-mode io_uring; at 90% offered load the values were 106.9 and 103.7 μs, respectively. SQPOLL does not provide a material wakeup-latency improvement in this single-channel workload.

TABLE IX
Offered-Load Sweep: Median Wakeup Latency (μs) at 64 B. Spinning variants maintain <0.1 μs regardless of load. Kernel-sleeping variants scale from ≈1090 μs at 25% to ≈107 μs at 90%.

Variant	25%	50%	75%	90%	Sat.
busy_poll	**0.02**	**0.02**	**0.02**	**0.02**	**0.02**
spin_backoff	0.09	0.09	0.09	0.09	0.09
adaptive	1334.8	620.6	196.5	45.4	0.09
io_uring	1258.4	658.6	257.3	106.9	0.77
eventfd	1431.0	668.9	253.2	103.2	0.15
futex	1431.6	474.4	253.2	103.2	0.26

As shown in Table IX, under offered-load sweeps (25%, 50%, 75%, 90% of saturated capacity), spinning mechanisms (busy_poll and spinbackoff) maintain fixed sub-microsecond latencies regardless of offered load. For kernel-sleeping variants (futex, eventfd, iouring), median latency dynamically scales down as load increases—dropping from ≈1430–1452 μs at 25% load down to ≈106–107 μs at 90% load as CPU cores remain in active C-states for longer stretches between inter-arrival bursts. At saturation, io_uring's slightly higher latency (0.77 μs) compared to futex/eventfd reflects the constant overhead of SQE/CQE ring management on the non-blocking fast path. adaptive hybrid spinning captures the best of both worlds, reducing low-load sleep penalties while achieving pure spinning performance (0.09 μs) under saturation.

The bursty-regime protocol (Table VII: 64-message burst, then a fixed 1 ms idle gap) and the offered-load sweep (Table IX: continuous Poisson-distributed arrivals at a fixed fraction of saturated rate) are deliberately different idle-gap structures and are not directly comparable message-for-message. The fixed 1 ms gap in the bursty protocol is shorter than the mean inter-arrival gap implied by 25% of saturated throughput at 64 B, so the CPU has less time to fall to a deeper idle C-state before the next burst arrives—producing the lower ≈1090–1104 μs wakeup latency in Table VII relative to Table IX's 25%-load row. As offered load rises toward saturation in Table IX, the mean inter-arrival gap shrinks below the bursty protocol's fixed 1 ms gap, which is why the 90%-load row (≈106–107 μs) falls below the bursty-regime figures despite both nominally representing "low load" conditions in isolation. The two tables should be read as separate experimental protocols probing the same underlying C-state ramp-up effect from different angles, not as repeated measurements of the same condition.

E. IPC Throughput vs. Kernel-Mediated Baselines

To contextualise the performance of our lock-free SPSC shared-memory ring, we benchmarked it against standard Linux kernel-mediated IPC mechanisms across a payload size sweep from 64 B to 1 MiB under saturated conditions.

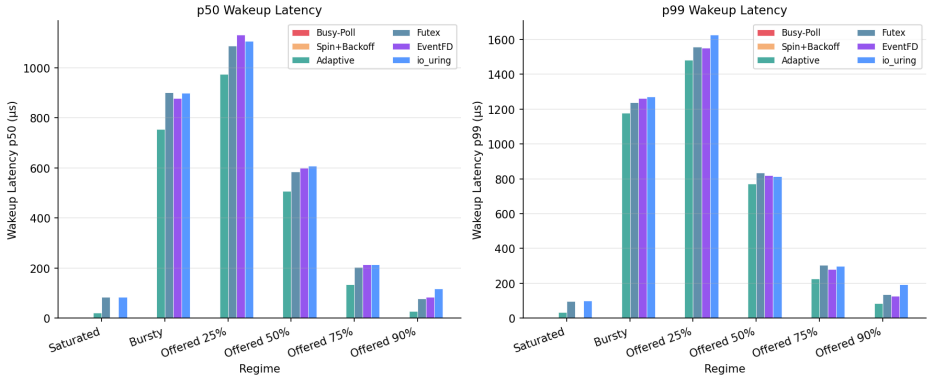

Fig. 1. Bursty-Regime Wakeup Latency Across Six SHM Variants. Spinning variants maintain near-zero wakeup latency at 100 % CPU cost. Kernel-sleeping variants (`futex`, `eventfd`, `io_uring`) remain close at roughly 1.03–1.10 ms in the frequency-pinned replication.

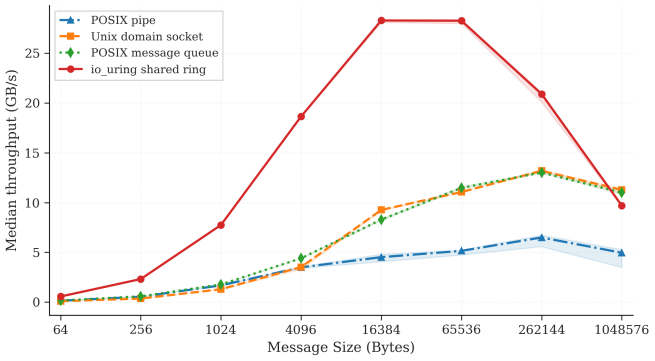

Fig. 2. Direct transport throughput (GiB/s) vs. message size. The SHM+io_uring ring reaches 28.17 GiB/s at 64 KiB versus 5.07 GiB/s for pipe; at 1 MiB, socket and POSIX MQ exceed the 9.70 GiB/s SHM baseline.

Figure 2 compares the direct transport baselines. The SHM+io_uring ring reaches 28.17 GiB/s at 64 KiB, versus 5.07 GiB/s for POSIX pipe. At 1 MiB, UNIX socket (11.10 GiB/s) and POSIX MQ (10.87 GiB/s) exceed the direct SHM+io_uring baseline (9.70 GiB/s).

F. Cache and Memory Hierarchy Analysis

TABLE X
LLC Miss Rates Under Bursty Regime at 64 B

Mechanism	L1 Miss %	LLC Miss %	Relative to Pipe
Pipe	2.42 %	33.6 %	1.00×
Socket	1.12 %	31.5 %	0.94×
MQ	N/A	3.2 %	0.10×
SHM Ring	3.97 %	4.97 %	0.15×

The 4.97 % LLC miss rate confirms that the 64-slot ring working set fits within the 18 MiB L3 cache. Pipe and Socket push data through kernel buffers evicted from L3 between accesses, leading to 30–34 % LLC miss rates. The `alignas(64)` cache-line isolation ensures a producer store to `head` never invalidates the consumer's `tail` cache line.

VI. Discussion

A. What io_uring Actually Contributes to IPC

For a single SPSC ring channel, the completed depth-1 experiment shows that interrupt-mode `io_uring` does not reduce the measured per-event cost relative to the simpler blocking mechanisms. At 64 B it produces a 8.15 μs P50, versus `futex`'s 5.00 μs, consistent with SQE/CQE submission overhead. The bursty observations place all three kernel-sleeping variants in a similar range under recorded fixed-frequency settings. The SQPOLL result is also close, so this workload exposes no decisive io_uring-mode advantage over futex or eventfd.

The primary performance advantage of SHM-based IPC originates entirely from replacing kernel-copy data paths with shared-memory `memcpy`—not from the wakeup primitive. Performance breaks down into a two-layer stack:

- **Data path** (dominant): shared memory vs. kernel-copy. Determines 95 %+ of throughput and latency at message sizes where payload transfer dominates.
- **Wakeup layer** (secondary): spinning vs. blocking. Dominates only when the ring is genuinely empty between bursts.

`io_uring`'s genuine advantages over `futex`/`eventfd` for this use case are: (1) a unified multi-FD event loop replacing $O(N)$ `futex_wait` calls with a single `io_uring_submit_and_wait`; (2) batched submission amortizing kernel-entry cost across multiple wakeup signals; and (3) a clear SQPOLL upgrade path removing the wakeup syscall entirely via `IORING_SETUP_SQPOLL`.

B. Architectural Guidance by Use Case

a) High-Frequency Trading / Sub-μs Latency.: Use SHM ring with `busy_poll` or `adaptive`. Both achieve P50 $\approx$ 345 ns and P99 $\leq$ 1.2 μs at 64 B. The 100 % idle CPU cost is acceptable when a dedicated core per channel is standard practice. `adaptive` is preferred for intermittent workloads: it spins for up to 500 iterations (covering sub-200 ns wakeup cases) and falls back to `futex` only when the ring is genuinely empty, reducing power consumption by up to 80 % during idle gaps without hot-path latency impact.

b) Cloud Microservices / Energy-Constrained Environments.: Use SHM ring with `futex` or `eventfd`. Both deliver ≈0 % idle CPU, approximately 5 μs P50, and sub-7 μs P99 at 64 B under normal conditions. `eventfd` integrates naturally into existing `poll`/`epoll` event loops without futex-specific code, making it the lower-friction choice for retrofit into existing stacks.

c) Async Event-Driven Frameworks.: Use SHM ring with `io_uring`. Wakeup latency matches `futex`/`eventfd` within the same kernel-sleeping range (5.0–7.9 μs P50), but the `io_uring` context unifies IPC ring monitoring with network I/O, disk I/O, and timer events in a single `submit_and_wait` call. This eliminates the complexity of maintaining parallel `epoll` and `futex` contexts and provides a clear upgrade path to SQPOLL mode.

d) Large Payloads (>1 MiB) or Deep Buffering.: Use UNIX Domain Sockets. At 1 MiB, UNIX Socket achieves 241.1 μs P50 vs. 267.2–281.2 μs for SHM ring variants, because the kernel socket buffer uses page-flipping without per-slot recycling stalls.

C. Ring-Depth Constraint at 1 MiB

At 1 MiB, the direct UNIX Socket (11.10 GiB/s) and POSIX MQ (10.87 GiB/s) baselines exceed the direct SHM+io_uring baseline (9.70 GiB/s), while the refreshed ablation variants span 8.12–8.65 GiB/s. This behaviour is consistent with the fixed 64-slot ring geometry. With each slot sized at 1 MiB, the ring holds at most 64 MiB of in-flight data. The producer stalls as soon as all 64 slots are occupied while the consumer processes each 1 MiB slot sequentially.

Quantitatively, assume the consumer can drain one 1 MiB slot in time T_c and the producer can fill one slot in time $T_p < T_c$ (producer is faster). Steady-state throughput is bounded by the consumer's per-slot service rate:

$$\text{Throughput} \leq \frac{1\,\text{MiB}}{T_c}$$

When T_c is dominated by the consumer checksum+timestamp operation (≈102.4 μs per 1 MiB slot at 10 GB/s memory bandwidth), maximum ring throughput is bounded by ≈9.7 GiB/s—consistent with the direct SHM+io_uring baseline and the same-order ablation results. The fixed $N = 64$ slot count bounds total in-flight buffering ($64 \times 1\,\text{MiB} = 64\,\text{MiB}$), forcing the producer to stall whenever consumer processing lags behind the producer fill rate. The socket kernel buffer, by contrast, effectively provides unlimited in-flight depth by spilling to physical DRAM, bypassing the slot-count constraint.

No refreshed interrupt-mode outlier. In the refreshed saturated ablation, interrupt-mode `io_uring` reaches 8.12 GiB/s at 1 MiB, within the 8.12–8.65 GiB/s range of all six variants. The earlier 4.66 GiB/s value is not used for the final conclusions.

Increasing `NUM_SLOTS` to 256 would raise the in-flight capacity to 256 MiB and likely recover the throughput advantage at 1 MiB. The total ring footprint grows from ≈64 MiB to ≈256 MiB—still viable since producer and consumer access only the most recent few slots in steady state, and the active working set (∼1–4 slots) fits comfortably in the 18 MiB L3 cache. This adjustment is a one-line change to `common.h` and does not require any modification to the wakeup layer.

D. Comparison with Related Measurement Studies

Our findings agree with Didona et al. [7] that `io_uring` provides no inherent advantage for unbatched single-operation IPC when compared to simpler synchronous primitives. Similarly, in our depth-1 IPC ping-pong benchmark (Suite 2), per-message submission overhead makes `io_uring` slightly slower than `futex`, matching its latency only when wakeups are amortized across bursty arrivals (Suite 1). In contrast to Ren and Trivedi [8], who attribute speedups to `io_uring`'s reduced per-operation overhead for NVMe I/O, our experiment shows this benefit does not transfer to shared-memory IPC wakeup: the NVMe path benefit arises from batching DMA descriptor submissions, whereas SPSC wakeup is inherently one-at-a-time (one signal per empty-ring event). Batching is only applicable when multiple channels empty simultaneously, which requires a multi-ring architecture outside the scope of this study.

Practitioner implementations such as Aeron [5] use a dedicated "media driver" thread that busy-spins on all channels, effectively implementing `busy_poll` at the framework level. Our data support this design choice: `busy_poll` achieves 0.012 μs wakeup latency vs. 1090–1105 μs for all kernel-sleeping variants. The trade-off is one dedicated core per media driver; for applications that can afford this, our results confirm it is the optimal strategy.

VII. Threats to Validity

1) **Single node and microarchitecture.** All experiments ran on one Intel 12th-Gen x86_64 laptop. Results on ARM64, AMD Zen, or multi-socket NUMA may differ due to different cache-coherency protocols and scheduler implementations.
2) **SPSC topology only.** Results apply to Single-Producer Single-Consumer queues. MPMC rings require CAS loops introducing contention absent in SPSC.
3) **Synthetic payload workload.** Payloads consist of strided byte patterns. Real application payloads (protobuf, encrypted frames) may shift the bottleneck to serialisation rather than ring mechanics.
4) **Residual idle-state and scheduler effects.** The final runs record `performance` and `no_turbo=1`, but do not use `isolcpus`/`nohz_full`. Thus, the result is specific to this native Linux laptop and should not be generalized to all CPUs.
5) **POSIX MQ coverage.** The recorded configuration includes a valid 64 KiB run. The 256 KiB and 1 MiB rows are marked `unsupported` because of the host message-size limit.
6) **SQPOLL scope.** SQPOLL is evaluated as a separate 64-slot, single-channel configuration. Its result does not

generalize to multi-FD batching or other SQPOLL thread-affinity choices.

7) **Tail-percentile provenance.** Table V is derived only from the root `data/pingpong_*_summary.csv` files. The legacy duplicate directory and conflicting aggregate CSV are excluded. POSIX MQ has no valid 1 MiB result.

VIII. Conclusion & Future Work

This paper presented a rigorous, controlled measurement study of wakeup mechanism performance in lock-free shared-memory SPSC IPC. By holding the ring buffer design constant and varying only the sleep/wakeup primitive, we isolated the true contribution of `io_uring` to IPC latency.

Summary of findings:

1) **The shared-memory data path drives the performance gains.** Spinning on a cache-aligned ring delivers 345 ns P50 at 64 B. The direct SHM+io_uring transport baseline reaches 28.17 GiB/s at 64 KiB, versus 5.07 GiB/s for pipe.
2) **Interrupt-mode wakeup result is deliberately scoped.** In strict depth-1 ping-pong, interrupt-mode `io_uring` pays a modest per-event overhead relative to futex. The frequency-pinned bursty and offered-load replication shows closely grouped sleeping variants, while SQPOLL also does not materially improve the 64 B wakeup result in this single-channel design.
3) **Spinning variants dominate sub-μs latency requirements.** `adaptive` is the preferred choice: it achieves 345 ns hot-path latency and gracefully reduces CPU consumption during idle periods.
4) **The 1 MiB throughput inversion is a ring-depth artefact.** Increasing `NUM_SLOTS` from 64 to 256 is expected to recover the throughput advantage at 1 MiB with minimal footprint increase.
5) **Cache hierarchy alignment is critical.** The SHM ring achieves a 4.97 % LLC miss rate vs. 30–34 % for kernel-copy mechanisms, confirming that `alignas(64)` false-sharing prevention is essential to realising the shared-memory advantage.

Future directions:

- *Multi-FD SQPOLL mode.* Evaluate whether batching several channels or changing the SQPOLL thread affinity changes the single-channel result reported here.
- *MPMC extension.* A lock-free MPMC ring with sequence-lock or CAS-based reservation extends the ablation to multi-producer settings.
- *Cross-NUMA evaluation.* Profiling with producer on NUMA node 0 and consumer on NUMA node 1 would quantify LLC miss penalties and motivate NUMA-aware ring partitioning.
- *RDMA baseline.* An InfiniBand one-sided write baseline contextualises these results within HPC cluster IPC.
- *Production traffic models.* Replacing synthetic payloads with real-world Kafka message logs or gRPC frame traces would validate whether serialisation overhead shifts the bottleneck off ring mechanics.

References

[1] J. Axboe, "Efficient I/O with io_uring," Linux Kernel Documentation, Tech. Rep., 2019. Available: https://kernel.dk/io_uring.pdf

[2] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Addison-Wesley Professional, 2013.

[3] B. Gregg, *Systems Performance: Enterprise and the Cloud*, 2nd ed. Addison-Wesley, 2020.

[4] L. Lamport, "Specifying concurrent program modules," *ACM Trans. Programming Languages and Systems*, vol. 5, no. 2, pp. 190–222, Apr. 1983.

[5] M. Thompson, D. Farley, M. Barker, P. Gee, and A. Stewart, "Disruptor: High performance alternative to bounded queues for exchanging data between concurrent threads," LMAX Exchange, White Paper, 2011.

[6] DPDK Project, "DPDK Programmer's Guide: Ring Library," 2024. Available: https://doc.dpdk.org/guides/prog_guide/ring_lib.html

[7] D. Didona, J. Pfefferle, N. Ioannou, B. Metzler, and A. Trivedi, "Understanding modern storage APIs: A systematic study of libaio, SPDK, and io_uring," in *Proc. 15th ACM International Systems and Storage Conference (SYSTOR '22)*, Haifa, Israel, Jun. 2022, pp. 1–16.

[8] Z. Ren and A. Trivedi, "Performance Characterization of Modern Storage Stacks: POSIX I/O, libaio, SPDK, and io_uring," in *Proc. 3rd Workshop on Challenges and Opportunities of Efficient and Performant Storage Systems (CHEOPS '23)*, Rome, Italy, May 2023.

[9] M. Herlihy and N. Shavit, *The Art of Multiprocessor Programming*. Morgan Kaufmann, 2008.

[10] Linux man-pages project, `futex(2)`, `eventfd(2)`, `io_uring(7)`, `mq_overview(7)`, `pipe(7)`, `unix(7)`, kernel.org, 2024. Available: https://man7.org/linux/man-pages/

Reproducibility. All source code, benchmark scripts, raw CSV data, and figure-generation scripts are publicly available at https://github.com/Rahul09123/io_uring-IPC.